

The Sound Half of POKEY

Four channels, nine bytes, and one flip-flop per voice. A reader's guide to the Atari C012294 — what each register does, why its pitch behaves backwards, and how a chip with no envelope generator made the sound of an arcade out of four numbers and an interrupt.

Atari C012294 · 1979 Reference machine: Tempest, 1.512 MHz Accompanies the PokeyLab demo

01	The output is one bit	02	AUDF, and why pitch runs backwards
03	AUDC: three fields in one byte	04	The polynomial counters
05	AUDCTL: changing the rules	06	The envelope is a table and an interrupt
07	The delta, and the wrap it depends on	08	Tempest's shot, dissected
09	Volume-only mode, and how a chip learned to talk	10	Reading the instruments
11	One question still open		

01 The output is one bit

Everything else is a rule about when that bit may change.

A POKEY channel's output is a single flip-flop — high or low, nothing in between. Every sound the chip has ever made is that bit toggling, or not toggling, at moments a divider picks.

There is no oscillator bank, no waveform table and no sample memory. The chip has **four independent channels**, each with a counter that decides when its flip-flop is allowed to flip, a field that decides what may veto the flip, and a four-bit volume. The four outputs are summed into one signal.

That sum is the reason a single channel never sounds as loud as you expect: the mix divides by all four channels at full volume — $4 \times 15 = 60$ — so one voice alone reaches a quarter of full scale whether or not the others are in use.

THE WHOLE SOUND HALF, IN NINE BYTES

Four `AUDF` registers, four `AUDC` registers, and one `AUDCTL`. That is the complete audio programming interface. Everything a program does to make sound is a write to one of those nine addresses — which is why a chip this simple needed software doing a great deal of work, and why the interesting part of arcade sound design lives in the code rather than the silicon.

WORTH KNOWING

The sound half is roughly a quarter of the part, and it is not what the chip is named after. **POKEY is POt and KEYboard** — officially the Pot Keyboard Integrated Circuit — because it reads eight paddle potentiometers and scans a keyboard matrix. It also drives a serial port, provides timers, and supplies the machine's random numbers. It is a general-purpose I/O chip that happens to sing.

02

AUDF, and why pitch runs backwards

The single least intuitive thing about the chip.

AUDF is not a note. It is a *divider* — the number of clock steps between one permitted toggle and the next. Two toggles make one complete cycle of a square wave, which is where the 2 comes from:

$$\text{pitch} = \text{clock} \div (2 \times (\text{AUDF} + 1))$$

So raising AUDF lowers the pitch. To make a sound rise, you subtract.

Because the register sits in the denominator, equal steps in AUDF are wildly unequal steps in pitch. A sweep written with a constant increment does not move at a constant rate — it produces a **hyperbola**, steep at first and flattening as it falls. That accelerating shape is characteristic of arcade sound, and it comes free from the arithmetic rather than from any deliberate curve.

It also means resolution is uneven. Near the top of a fast sweep a single step of AUDF can be twenty hertz or more, while at the bottom it is a fraction of one — audible as a staircase in the high register, invisible in the low.

THE RELOAD ADDEND

The counter reloads with AUDF plus a small constant — +1 on the divided clocks, +4 for a channel taking the base clock directly, and +7 for a joined 16-bit pair on the base clock. These are the hardware's own reload delay, not a fudge factor, and they are confirmed by the manufacturer's documentation.

AUDC: three fields in one byte

A packed byte, and the arithmetic is legible once you know the boundary.



AUDF's value is a quantity: 16 means sixteen. AUDC's is not — 162 means distortion 5 and volume 2, and nothing about the number says so.

Because distortion occupies the top three bits, the byte reads as simple arithmetic:

$$\text{AUDC} = \text{distortion} \times 32 + (\text{volume-only} ? 16 : 0) + \text{volume}$$

Tempest's player shot uses \$A2 — that is $5 \times 32 + 2$, distortion 5 at volume 2.

Reading one back is the same rule in reverse: divide by 32 (or shift right five) for the distortion, test bit 4 for volume-only, take the remainder mod 16 for the volume.

THE HEX SHORTCUT

Write the byte in hexadecimal and the fields fall out almost by inspection, because the nibble boundary lands neatly. **The low hex digit is the volume, always.** The high digit is the distortion doubled — halve it and you have the code. And since the volume-only flag is squatting in the bottom of the high nibble, **an odd high digit means volume-only is on.**

So \$A2 is volume 2, distortion 5, oscillator running. \$1A is volume 10, distortion 0, and volume-only set — a channel being used as a converter rather than an instrument.

WHAT THE DISTORTION FIELD SELECTS

It chooses which polynomial counter must agree before the flip-flop is allowed to toggle. Two codes give the same result as two others, and the reason is worth knowing: the low bit of the field **bypasses the second counter stage**, so 1 and 3 are both the plain 5-bit counter, and 5 and 7 are both an ungated square.

THE DISTORTION FIELD

CODE	GATED BY	WHAT IT SOUNDS LIKE
0	5-bit then 17-bit	Coarse, gritty noise
1, 3	5-bit	Rough but still musical
2	5-bit then 4-bit	The roughest pitched texture available
4	17-bit	True noise — explosions, wind
5, 7	nothing	Pure tone — a square wave
6	4-bit	Short, buzzing growl with a pitch of its own

WRITING ONE, AS A PROGRAMMER

You never compute an AUDC literal. You pick a texture and a loudness, and the byte assembles itself — which in code means naming the distortions once and adding volume at the point of use, so a voice reads `Noise + 10` rather than `138`.

And if you are changing one field at runtime, read-modify-write, masking the field you are not touching. A naive write of a bare volume **destroys the distortion**, because they share the byte.

THE TRAP

A bare distortion value is silent. `192` looks like "distortion 6" and is — with volume 0. Nothing comes out. This is not hypothetical: it is exactly what made nine sounds inaudible in the first sound-bank files written for this project. You never write a distortion; you write `distortion × 32 + volume`.

YOU CANNOT READ IT BACK

POKEY's audio registers are **write-only**. The same nine addresses decode to entirely different registers on a read — the paddle inputs, the keyboard code, the serial and interrupt status, and `RANDOM`. A physical POKEY has no way to tell you what is in AUDC1; the only record is whatever the program remembers writing. The lab can show you the byte because its emulation keeps its own copy, which is a fair thing for an instrument to do but is a view the hardware never offered anyone.

The polynomial counters

The idea the chip is named for — and it is not random.

POKEY's noise is not noise. It is a handful of shift registers running a fixed, repeating sequence — and a channel hears a pitch because it samples that sequence at a rate you choose.

Each counter is a linear-feedback shift register: exclusive-OR two of its own bits, shift right, push the result back in at the top. Chosen well, the taps make the register visit every non-zero state before repeating — which is why the periods are $2^n - 1$ and not 2^n . The all-zero state is a trap the counter can neither enter nor leave.

WIDTHS, AND WHAT WIDTH DECIDES

COUNTER	POLYNOMIAL	REPEATS EVERY	CHARACTER
4-bit	$x^4 + x^3 + 1$	15	Rekurs constantly — reads as a tone
5-bit	$x^5 + x^3 + 1$	31	Coarse, still musical
9-bit	$x^9 + x^5 + 1$	511	Buzzy, audibly periodic
17-bit	$x^{17} + x^{12} + 1$	131,071	No audible recurrence — true noise

Same mechanism, opposite sound, and the only difference is register width. A short sequence recurs quickly enough for the ear to hear a period, which is a pitch. A long one does not, which is noise.

TWO CLOCKS, AND THAT IS THE WHOLE TRICK

The counters **free-run at the base clock**, continuously, regardless of what any channel is doing. A channel looks at one of them once per divider period. So what you hear is not the sequence — it is the sequence *strobed* at whatever rate `AUDF` sets.

Turn the divider and the sampling points move while not one bit of the sequence changes. That single observation explains the entire distortion field faster than its name does.

A SECOND JOB

The 17-bit counter is also the machine's random number generator. The CPU reads its top eight bits from the `RANDOM` register — and the same `AUDCTL` bit that shortens it for sound shortens it for randomness too. The noise you are listening to is the source of every random decision the machine makes.

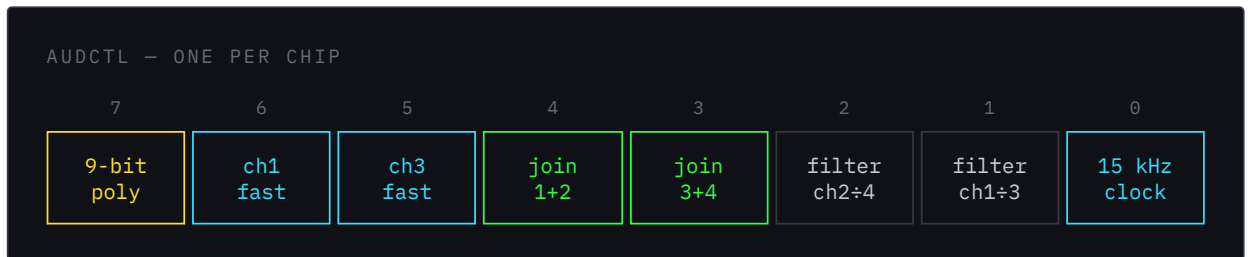
OPEN QUESTION

The tap positions above are the standard maximal-length polynomials. They have not been verified bit-for-bit against real hardware, and the manufacturer's documentation does not give them. The structure is certainly right; the exact sequence is an assumption.

05

AUDCTL: changing the rules

One register that alters what the other eight mean.



Eight bits, but really four ideas. **Clock selection** — bit 0 drops all four channels from base $\div 28$ to base $\div 114$, while bits 5 and 6 do the opposite for one channel each, skipping the divider entirely. **Pitch resolution** — bits 3 and 4 glue two channels into one 16-bit counter, trading a voice for precision. **High-pass filters** on channels 1 and 2, each clocked by another channel, so the filter costs you the channel that drives it. And **noise length**, which shortens the long counter to nine bits.

THE CLOCK IS NEVER 64 KHZ

POKEY's so-called "64 kHz clock" is not an oscillator. It is the chip's own clock input divided by 28, so it moves with the machine. On an Atari home computer at 1.78979 MHz that works out to **63.9210 kHz** — never the round number the name implies. On Tempest, clocked at 1.512 MHz, the same divider gives **54,000 Hz**, and every pitch on the chip moves with it.

This is why a sound designed on one machine and played on another is not slightly off but in a different key.

JOINING A PAIR

A 16-bit join makes the lower-numbered channel the *low byte*. It stops driving the mix altogether — its whole job becomes clocking its partner, and the audible voice is the higher channel. A joined channel therefore reports no pitch of its own, and its volume and distortion settings stop mattering.

This is the complete answer to a question the lab makes easy to stumble into: join channels 1 and 2 while a sound is playing on channel 1, and the sound goes silent. Nothing broke. Channel 1 is not a voice any more.

HISTORICAL NOTE

Tempest writes zero to AUDCTL on both of its POKEYs. Every feature in this chapter is switched off in the finished game. All of its variety comes from rewriting AUDF and AUDC on a timer instead — which is the subject of the next three chapters.

06

The envelope is a table and an interrupt

There is no envelope generator. There is arithmetic, run 220 times a second.

A Tempest sound effect is four numbers per register and a routine that walks them a couple of hundred times a second. That is the entire sound engine.

POKEY holds whatever was last written to it until something writes again. It has no attack, no decay, no sustain and no release — those are all things a program does by rewriting registers on a timer. What Tempest has is an interrupt routine that walks a table of four-byte steps and pokes the results at the chip.

THE STEP RECORD

ONE STEP — THE SHOT'S AUDF, AS STORED IN ROM

VALUE	INNER	DELTA	OUTER
16	1	+7	32
<i>What the register starts at.</i>	<i>Ticks to hold before each change.</i>	<i>What is added each time.</i>	<i>How many times round.</i>

Read it as a sentence: write *Value*, hold it for *Inner* ticks, add *Delta*, and do that *Outer* times. The field order above is the order the interrupt fetches them in, not an order chosen for a display.

Two quantities fall straight out of the four. A step occupies $\text{Inner} \times \text{Outer}$ ticks, so its duration in seconds is that divided by the tick rate. And the value moves $\text{Outer} - 1$ times, which is the first of three details that decide whether a reproduction is faithful.

THREE DETAILS THAT DECIDE FAITHFULNESS

1. It applies $\text{Outer} - 1$ deltas, not *Outer*. The counter that reaches zero advances to the next step instead of adding. Tempest's shot therefore ends at $16 + 7 \times 31 = 233$, not 240 — about two hertz at the bottom of the sweep, and the difference between a model derived from the interrupt handler and one derived from a plausible reading of it.
2. AUDF and AUDC have separate chains, and they run independently. Different lengths, different step counts, different deltas. The sound lasts

as long as the longer of the two — so lengthening one can leave a tail after the other has finished.

3. **Registers wrap rather than clamp, and that is used deliberately.** The interrupt's add is plain 8-bit arithmetic with no guard. Clamping is the intuitive reading and it is wrong: it parks Tempest's tube-descent effect at divider 0, a 16 kHz squeal, where the hardware runs off the bottom, through zero, and lands on 255 as a low rumble.

WHAT THE TICK RATE DOES, AND DOES NOT, CONTROL

The tick rate sets **duration and nothing else** — pitch comes from the divider and the clock. At 220 Hz the interrupt is running several times per video frame, which is how a 32-step envelope fits into 0.145 seconds instead of half a second. Halve the tick rate and the same sweep covers the same pitches over twice as long.

WHY IT IS A STAIRCASE

The sweep graph in the lab draws a stair, not a slope, and that is not a simplification. The register holds a value for `Inner` ticks and then jumps. Nothing glides. Each stair's *width* is `Inner` and its *height* is `Delta` — the four fields are literally the shape on screen.

07

The delta, and the wrap it depends on

On AUDC the arithmetic is four bits wide, and that changes everything about it.

Chapter 6's rule is the same for both chains. What differs is the width of the arithmetic. On AUDF the add is masked to a full byte. On AUDC it is masked into the volume nibble, with the top four bits put back untouched:

```
AUDC ← (value & 0xF0) | ((value + delta) & 0x0F)
```

The distortion nibble survives every tick, by construction.

A sound's timbre is fixed for its entire life. Whatever distortion it starts with, it finishes with. Only the loudness ever sweeps.

That is a real statement about how these sounds work, not an implementation detail. There is no fade from a tone into noise anywhere in Tempest — not because nobody thought of it, but because this engine structurally cannot express it. Changing texture mid-sound costs a whole second step.

ONLY DELTA MOD 16 DOES ANYTHING

Since the result is masked to four bits and the nibble wraps rather than clamping, a delta and that delta plus sixteen are indistinguishable. Two consequences catch everyone:

- **-8** and **+8** are the same edit. Both take a volume of 2 to 10.
- **+16**, **-32** and **+64** all do nothing. The volume never moves.

So a field offering -64 to 64 offers 129 numbers and sixteen behaviours. The useful way to read what is left is that the volume walks a ring of sixteen levels, stepping `delta mod 16` places each tick, and returns to where it started after:

$$\text{cycle} = 16 \div \text{gcd}(\text{delta}, 16)$$

WHAT A DELTA IS WORTH ON AUDC

DELTA (MOD 16)	CYCLE	CHARACTER
± 8	2	Hard alternation — the tightest cycle there is
± 4 , ± 12	4	Fast four-level churn
± 2 , ± 6 , ± 10 , ± 14	8	Tremolo
odd	16	Walks every level — the closest thing to a ramp
0, ± 16 , ± 32	—	Nothing moves

WHY ± 8 IS SPECIAL: IT IS A BIT TOGGLE

On a four-bit field that wraps, adding or subtracting 8 is *exactly* the same as flipping the top bit of the nibble:

$$(v \pm 8) \bmod 16 == v \text{ XOR } 8$$

2 becomes 10, 10 becomes 2, and it never settles.

So a delta of -8 does not decrement anything. It is a **one-bit square-wave tremolo**, and it is the only delta that produces one — every other value walks a longer ring. That reframes what looks like a fade that happens to wrap into something deliberate.

INNER IS THE SPEED CONTROL, AND THE ONLY ONE

Combine the cycle with the hold time and you get the actual modulation frequency:

$$\text{modulation Hz} = \text{tick rate} \div (\text{cycle} \times \text{Inner})$$

Delta chooses the shape of the volume cycle; Inner chooses its speed. They are not interchangeable, and that is a consequence of the wrap. You cannot slow a tremolo down by using a smaller delta — a delta of 4 instead of 8 does

not move more gently, it moves through a four-step ring instead of a two-step one, which is a different texture at a similar rate. Only Inner changes the rate.

The same applies to a genuine fade. `delta -1, Inner 1` walks 15 down to 0 over fifteen ticks — about 68 ms, which is a click rather than a fade. At `Inner 8` the same fifteen steps take half a second.

ON AUDF THE WRAP IS USED TOO

The mask there is a full byte rather than a nibble, so a delta has 256 distinct behaviours and every value in a normal range is meaningful. But the wrap is still load-bearing: the tube-descent effect runs the divider off the bottom of the byte, through zero, and lands on 255 — the lowest pitch the chip has — which is why it ends as a rumble instead of a squeal.

Tempest's shot, dissected

Eight numbers, byte-for-byte from the ROM, and what each one is doing.

The player's shot is the most-heard sound in the game and the whole engine in miniature: two chains, one step each, four numbers apiece.

THE SHOT, AS THE ROM STORES IT

CHAIN	VALUE	INNER	DELTA	OUTER	WHAT IT DOES
AUDF	16	1	+7	32	Pitch falls 1588 → 115 Hz
AUDC	162 (\$A2)	1	-8	32	Distortion 5, volume toggling 2 ↔ 10

THE FIRST NINE TICKS

WHAT THE INTERRUPT WRITES, TICK BY TICK									
TICK	0	1	2	3	4	5	6	7	8
AUDF	16	23	30	37	44	51	58	65	72
PITCH HZ	1588	1125	871	711	600	519	458	409	370
VOLUME	2	10	2	10	2	10	2	10	2

AUDF rises by 7 every tick — so the pitch falls, and unevenly volume never settles

Two things are visible there that no description quite conveys. The pitch column collapses — 463 Hz between the first two ticks, 39 Hz between the last two, from an identical +7. That is chapter 2's hyperbola, happening. And the volume column never settles anywhere; it is a square wave in its own right.

THE NUMBERS, ONE AT A TIME

Volume 2 as the base decides what the toggle swings between. Starting at 2 gives 2 ↔ 10: a 5:1 amplitude swing that never reaches silence, averaging 6 out of 15 so the shot sits in the mix without dominating it. Had the base been 8 it would swing 8 ↔ 0, chopping to full silence half the time — a much harsher, more mechanical gating.

Inner 1 is the fastest the chain can run. With a cycle of 2 at a 220 Hz tick that is a **110 Hz** modulation — down in the audible range itself, so you do not hear it as pulsing but as roughness and sidebands on the tone. At **Inner 4** it would drop to 27 Hz and start to sound like a stutter.

Outer 32 is simply AUDF's length. The two chains are deliberately the same size, so the volume thrash starts and stops exactly with the pitch sweep. 32 ticks at 220 Hz is 0.145 s. An aside that falls out: 31 deltas is an odd count, so the shot ends on volume 10 — its louder level — at its lowest pitch.

WHAT THE DESIGN AMOUNTS TO

Note what the shot is *not* doing. Distortion 5 is **pure tone**, the cleanest waveform the chip has. The grit is not coming from a noise distortion — codes 0, 2, 4 and 6 were all available and none was used. The design is instead *clean oscillator, roughed up by amplitude modulation*, which gives a shot that stays pitched and readable as it falls while still sounding aggressive.

And it is the cheapest way to get there. POKEY has no tremolo and no amplitude envelope, and the delta cannot touch the distortion field because it is masked. So: one step, four numbers, no second channel — and the wrap that looks like an oversight is doing the work.

HONESTY

Nobody left comments in the ROM. This is what the numbers demonstrably do, plus what the hardware's constraints made cheap — not documented intent. It is possible the programmer tried values until it sounded right and -8 won. What is certain is that this is not a decay written by someone who forgot to clamp: -8 at Inner 1 is the single most effective roughness the register can produce, and it would be a remarkable accident.

Volume-only mode, and how a chip learned to talk

The most interesting thing POKEY does is stop being POKEY.

With AUDC bit 4 set, the divider, the counters and the filters are all cut out of the path. Whatever value is written to the four-bit volume field appears at the output. Sixteen levels, rewritten as fast as the processor can manage.

This was the machine's **only sample path**, and in 1982 Software Automatic Mouth used it to speak. S.A.M. was a formant synthesiser running on a 6502 — no recordings, no sample memory. It modelled a mouth: an impulse train for the vocal folds and a set of resonators standing in for the throat, tongue and lip cavities. The output had nowhere to go but this nibble.

A POKEY that is speaking is a POKEY switched off.

WHY PERIOD SPEECH SOUNDS LIKE PERIOD SPEECH

Two limits, both audible. The first is the four-bit quantisation. The second is the interrupt rate the processor could sustain *while also doing the synthesis* — and that rate decides which formants survive at all.

WHAT THE INTERRUPT RATE CAN CARRY

RATE	HIGHEST FREQUENCY	CONSEQUENCE
------	-------------------	-------------

4 kHz	1,800 Hz	Loses the third formant — <i>ng</i> collapses onto <i>n</i> , and the sibilants go with it
-------	----------	--

8 kHz	3,600 Hz	Keeps all three formants
-------	----------	--------------------------

The same trick appears elsewhere in the era — the Commodore 64's volume-register digi works on identical principles. What sounds in hindsight like a stylistic choice was an arithmetic constraint.

10

Reading the instruments

Four displays, each showing a stage the others cannot.

The channel strips set the registers; the instruments show what the registers did. Between them sits the mechanism, which is why there is more than one display.

THE REGISTER STRIP — THE CAUSE

All nine bytes, live, as bits, with AUDC's three fields coloured apart. Every other display shows an effect; this one shows what the chip was actually told. It is the only place the arithmetic of chapter 3 is visible rather than asserted — set distortion 5 and volume 2 and watch `101` and `0010` assemble into `$A2`. It is a read-out, not a control: every bit in it already has a control elsewhere.

THE SCOPE — THE RESULT

The chip's actual output, drawn as a waveform. Triggered on a rising zero crossing so a steady tone stands still, which matters more than it sounds: drawn untriggered, a stable waveform slides sideways at the beat frequency between the audio and the display, and looks like the chip is doing something it is not. Three pictures teach the distortion field — a pure tone is a square, the 5-bit counter is a short pattern you can nearly count, and the 17-bit never visibly repeats.

THE POLY TRAINS — THE MECHANISM

The counters drawn as the bit sequences they are, with a marker where each one repeats and ticks beneath showing where the current channel samples. The 4- and 5-bit counters wear their repeat marker plainly; the 9- and 17-bit ones repeat far off the edge of the display, which is exactly why they sound like noise instead of a tone. Nothing here is a control — the sequences are fixed properties of the design.

THE SWEEP GRAPH — THE ENVELOPE OVER TIME

Pitch against time for the whole sound, drawn as a *staircase* because that is what it is: the interrupt rewrites the divider and the value holds until the next tick. The wide steps at the left and the smooth line at the right are the

hyperbola of chapter 2, made visible. A green target box marks where reference recordings of a real cabinet say the sweep should end, and the axes are scaled to keep that box on screen even when the current settings are nowhere near it.

All four are drawn from the same arithmetic that drives the audio. If the emulation were wrong, the pictures would be wrong in exactly the same way — none of them can reassure you about something the speaker is contradicting.

One question still open

Where the reproduction and the recordings disagree.

Tempest's player shot is transcribed byte-for-byte from the game's own ROM: AUDF starting at 16, adding 7 every tick, over 32 steps. Played through the emulation at the machine's real clock, it does not match recordings of the original cabinet.

THE SHOT, AS WRITTEN VERSUS AS RECORDED

QUANTITY	FROM THE ROM	MEASURED	VERDICT
Starts at	1,588 Hz	900 - 1,100 Hz	no
Ends at	115 Hz	140 - 195 Hz	no
Lasts	0.145 s	0.136 - 0.184 s	ok
Falls by	13.8 : 1	5.7 - 6.6 : 1	no

The duration is right, so the tick rate is settled — and with it everything in chapters 6 and 7 that depends on it, including the AUDC side of the shot. The *fall ratio* is the sharpest of the disagreements, because it contains no clock term at all — $(\text{AUDF_end} + 1) \div (\text{AUDF_start} + 1)$ — so no clock setting can rescue it. It is wrong at every clock, which places the problem in the transcribed data or the recordings rather than in the emulation.

The hardware side is now fully sourced: the sound part is clocked at 1.512 MHz, the same as the CPU; it divides that by 28; and the game selects that divided clock. So the remaining possibilities are narrow — either the first field of the step table is not what it appears to be, or the recordings are not capturing what they are assumed to be.

WHY THIS IS IN THE DOCUMENT

A tool that only displayed agreement would be worthless. The target box exists so the disagreement cannot be quietly forgotten, and the axes are deliberately scaled to keep it visible. This is what the instrument is for.

Appendix

REGISTER SUMMARY — THE SOUND HALF

REGISTER	COUNT	CONTENTS
AUDF1–4	4	Divider. Pitch = clock ÷ (2 × (AUDF + 1))
AUDC1–4	4	Distortion (bits 7–5), volume-only (bit 4), volume (bits 3–0)
AUDCTL	1	Clock selection, 16-bit joins, high-pass filters, poly length

THE STEP RECORD — BOTH CHAINS

FIELD	MEANING	ON AUDF	ON AUDC
Value	Starting register content	A divider, 0–255	distortion × 32 + volume
Inner	Ticks held before each change	Stair width on the sweep	Tremolo speed
Delta	Added each time, wrapping	Masked to a byte — 256 behaviours	Masked to a nibble — 16 behaviours
Outer	Times round; applies deltas	Step lasts $\text{Inner} \times \text{Outer}$ ticks	

CLOCKS, BY MACHINE

MACHINE	CHIP CLOCK	÷ 28	÷ 114
Atari 400/800, NTSC	1.78979 MHz	63,921 Hz	15,700 Hz
Tempest arcade	1.512 MHz	54,000 Hz	13,263 Hz

TERMS

LFSR — linear-feedback shift register, the circuit behind each polynomial counter. **Formant** — a resonant peak in the vocal tract; three of them are enough to distinguish most vowels. **Nibble** — four bits, half a byte, and here the entire dynamic range of a voice. **Tick** — one firing of the interrupt that walks the envelope table; on Tempest, 220 of them a second.

SOURCES AND CAVEATS

Register behaviour, the distortion table, the reload addends and the derived clock frequencies are drawn from the manufacturer's C012294 documentation. Envelope structure, the $\text{Outer} - 1$ rule, the nibble mask and the eight transcribed sound effects come from Tempest's own disassembled source, verified against the interrupt handler. The delta arithmetic in chapter 7 is derived from that mask and checked against the emulation for every delta the lab accepts. Polynomial tap positions are the

standard textbook values and remain unverified against hardware. The clock disagreement in chapter 11 is unresolved at the time of writing, and is stated here as an open question rather than a finding.